

HyMeKo: A Canonical Signed-Hypergraph IR for Structurally Accountable CPS Engineering

Aiko (Claude Code) for Dr. Csaba Hajdu

2026-06-16

Abstract

Cyber-physical-systems (CPS) engineering accumulates many partial artifacts — block diagrams, kinematic models, requirements traces, neural surrogates — that drift out of mutual agreement. HyMeKo proposes the opposite discipline: a single canonical *signed-typed hypergraph* intermediate representation (IR), treated as the one source of truth, that compiles to many faithful targets under a validation gate. From one `.hymeko` source the framework emits 2D/3D graph views, a SysML lens, a kinematic view, a PyTorch `nn.Module` (as a dataflow transform), a DOT requirements-traceability graph, and a canonical store (HIVE) answerable by typed queries — the “many lenses, one source” property. Structural accountability is enforced by a validation gate plus *structural parity*: emitted neural submodules must correspond one-to-one with IR layers, and a deliberately “broken twin” fails the gate and refuses to emit. The *infrastructure is the contribution* of this article. We also report a single empirical case study — a Bochner-wrapped hypergraph-convolution backbone on vision — which improves over a previously-falsified bare-sum aggregator across three settings, but which we present honestly as **preliminary** and **single-seed**: it does *not* overturn the standing falsification of the differential-geometric vision model at the scales we can afford. The new contribution to parity tooling — a typed family of hypergraph generators (Steiner triple systems, sunflower / Δ -systems, complete r -uniform hypergraphs) with 22 passing in-crate tests — is described in full.

1 Introduction and motivation

A CPS model is rarely a single object. In practice an engineering team carries a block-diagram view, a kinematic model, a requirements spreadsheet, a simulation harness, and increasingly a learned surrogate, each maintained by a different tool and a different person. These artifacts are *supposed* to describe one system, but nothing forces them to agree. They drift. A requirement is amended in the spreadsheet but not the block diagram; a neural surrogate is retrained on a topology the kinematic model no longer matches. The cost of this drift is paid at integration time, when the artifacts are reconciled by hand, or worse, in the field, when they are not.

HyMeKo takes the position that there should be *one accountable model*, not N drifting artifacts. The canonical object is a signed-typed hypergraph IR written in a small textual surface syntax (a `.hymeko` source). Everything else — diagrams, lenses, kinematic views, neural modules, requirement traces, queryable stores — is a *compilation target* produced from that IR by a deterministic transform. When the IR changes, every target is regenerated from it; the targets cannot silently diverge because they are never authored independently. This is the same discipline a compiler imposes between source and object code, applied to systems engineering artifacts.

The contribution of this article is the *infrastructure* that makes this discipline concrete: the IR and its signed incidence algebra (Section 3), the multi-target compiler (Section 4), the validation gate and structural-parity check that make “accountable” a testable property rather than a slogan (Section 5), a typed family of hypergraph generators used for parity testing (Section 6), and the P-graph methodology available in the framework (Section 7). We then report a single empirical

vision case study (Section 8), which we are careful to frame as preliminary and as a study of the IR’s neural *target*, not as a vision result that overturns prior negative findings.

2 The HyMeKo IR: signed-typed hypergraphs

A hypergraph generalizes a graph by allowing an edge to connect any number of vertices [1]. HyMeKo’s IR is a *signed, typed* hypergraph: each vertex and hyperedge carries a type, and each incidence (the membership of a vertex in a hyperedge) carries a sign. The sign distinguishes, for example, an activating from an inhibiting participation, or a positive from a negative contribution to a contract — the same role signs play in signed graphs and signed lateral interactions.

Signed incidence. Let V be the vertex set and E the hyperedge set, with $|V| = n$ and $|E| = m$. The signed incidence is a matrix $H \in \{-1, 0, +1\}^{n \times m}$ where $H_{ve} = 0$ when vertex v is not in hyperedge e , and $H_{ve} = \pm 1$ encodes the signed participation otherwise. Let $\bar{d}$ denote the mean hyperedge degree (the average number of vertices per hyperedge). The signed incidence carries the full combinatorial content of the IR; the two ways it is unfolded into a node-level operator have very different cost.

Star expansion (signed Levi incidence). The star expansion introduces one auxiliary node per hyperedge and connects it to each member vertex, reproducing the (signed) Levi / incidence bipartite graph. Each hyperedge of degree d contributes d nonzeros, so the total number of nonzeros is

$$\text{NNZ}_{\text{star}} = \sum_{e \in E} d_e = m \bar{d}, \quad \text{i.e. } O(|E| \cdot \bar{d}).$$

The expansion is *sparse*: its cost is linear in the mean degree.

Clique expansion. The clique expansion instead replaces each hyperedge by a clique over its member vertices. A hyperedge of degree d then contributes $O(d^2)$ edges, so

$$\text{NNZ}_{\text{clique}} = \sum_{e \in E} O(d_e^2) = O(|E| \cdot \bar{d}^2),$$

quadratic in the mean degree — a *dense* expansion.

Honest trade-off. Star expansion uses fewer nonzeros (NNZ) but requires more dimensions (the auxiliary hyperedge nodes); the sparse structure is therefore preferred. The IR keeps the signed incidence as its canonical form and chooses the star expansion when a node-level operator is needed, accepting the extra dimensions in exchange for the linear-in- $\bar{d}$ sparsity.

3 Multi-target compilation: many lenses, one source

From a single `.hymeko` source the framework compiles a family of targets, each a faithful *lens* on the same IR. Figure 1 shows the fan-out.

2D / 3D graph views

Layout-oriented renderings of the hypergraph for inspection and editing.

SysML lens

A projection of typed vertices and hyperedges onto SysML [7] constructs, so the same IR can be read by a systems engineer in familiar notation.

Kinematic view

A mechanism/linkage projection for the CPS-physical part of the model.

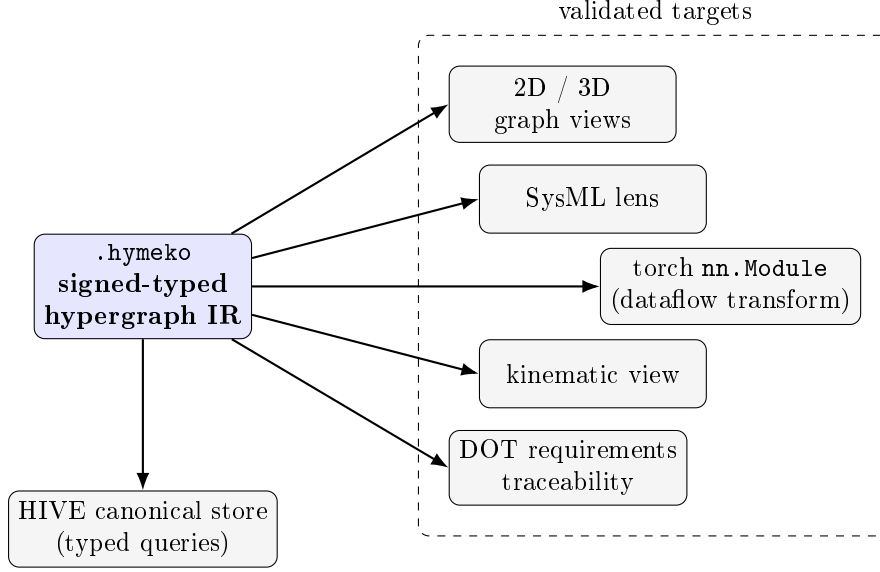


Figure 1: One source, many targets. A single `.hymeko` IR fans out to graph views, a SysML lens, a kinematic view, a PyTorch `nn.Module`, a DOT requirements-traceability graph, and the HIVE canonical store. Each target is a deterministic compilation of the same IR, so targets cannot drift independently.

PyTorch `nn.Module`

The IR is compiled, as a dataflow transform, into a neural module whose computation graph mirrors the hypergraph. This is the “neural target” exercised empirically in Section 8.

DOT requirements traceability

A Graphviz DOT graph linking requirements to the IR elements that satisfy them.

HIVE canonical store

A canonical store of the IR answerable by typed queries, providing a persistent, queryable source of truth.

Because every target is generated from the IR, “many lenses, one source” is not a wish but a structural guarantee: there is exactly one editable artifact.

4 Structural accountability

“Accountable” is made testable by two mechanisms.

Validation gate. Compilation is gated. An IR that fails validation does not emit any target. This turns malformed or internally-inconsistent models into a hard failure at compile time rather than a silent, plausible-looking but wrong artifact downstream.

Structural parity. For the neural target, accountability is sharpened into a one-to-one correspondence: each emitted neural submodule must correspond to exactly one IR layer, and vice versa. The emitted `nn.Module` is therefore not an opaque approximation of the IR — it is a structural image of it, checkable element by element.

The broken-twin test. To show the gate has teeth, the framework includes a deliberately corrupted “broken twin” of a valid IR. The twin fails the validation gate and *refuses to emit*. This is the positive evidence that the gate rejects what it should: a model that cannot be made

structurally accountable does not produce a target at all, rather than producing one that quietly disagrees with its source.

5 Hypergraph generators: a parity contribution

Testing structural parity requires a supply of hypergraphs with known, combinatorially exact structure. The framework provides a typed family of generators, unified behind a single typed enum (`HypergraphGenerator`), so that a caller selects a construction by a typed variant rather than by a string-typed mode. The family covers:

- **Steiner triple systems** $S(2, 3, n)$, a classical design in which every pair of points lies in exactly one block (triple). Three constructions are provided according to n :
 - the canonical **Fano plane** for $n = 7$;
 - the closed-form **Bose construction** for $n \equiv 3 \pmod{6}$;
 - a **most-constrained-pair (MRV) backtracking** search for $n \equiv 1 \pmod{6}$.
- **Sunflowers** (Δ -systems): families of sets sharing a common core, used as a controlled-overlap stress structure.
- **Complete r -uniform hypergraphs** $K_n^{(r)}$: all r -subsets of an n -set, a dense reference structure.

The single enum dispatches to the correct construction, so adding a new generator means adding a variant, not a new free-function family. **22 in-crate tests pass** for this generator family.

6 P-graph methodology

The framework includes a Process-graph (P-graph) methodology, *available as infrastructure* (we do not claim it has been applied to the vision study). Specifically it provides:

- the Friedler P-graph axioms **A1–A5** [6];
- **Maximal Structure Generation (MSG)** and **Solution Structure Generation (SSG)**;
- the **Accelerated Branch-and-Bound (ABB)** algorithm; and
- reachability rules over the process structure.

This methodology is validated against the worked examples in the Friedler P-graph textbook. We state it here as a capability of the framework, not as a component of the empirical study below.

7 Empirical case study: a hypergraph neural target on vision

All results in this section are PRELIMINARY and single-seed where so noted. They concern the IR’s neural target, not a claim about the vision model as a whole.

Architecture. The backbone is a Bochner-wrapped hypergraph-convolution model with three branches (*walk*, *polygon*, *triangle*). Prior work *falsified* the bare-sum aggregator that simply summed the branches. The present study replaces it with a learned- α_k branch mixer, a highway gate of the form $T \cdot H(m) + (1 - T) \cdot s$, and a top-down cross-scale pyramid. We compare this *upgraded* model against the *baseline* (bare sum) in three settings.

MNIST classification (3 paired seeds, 4 epochs, CPU). Here three paired seeds are used; we report the mean and a paired standard deviation (pstd). The upgraded model improves the mean test accuracy by +11% relative and reduces variance by roughly 5 $\times$, at a modest parameter cost.

Cluttered-MNIST detection. Two detection settings are reported, both single-seed: a *toy* run (400 training images, 8 epochs) and a *scale* run (2000 training images, 20 epochs, GPU). The proxy metric is mAP50_proxy. In the scale run the upgraded model is **still climbing at epoch 20**: its last three epoch values are 0.117, 0.126, 0.135, so 0.135 is a not-yet-converged final value, not a plateau.

Setting	Model	Metric	Value	Params
MNIST cls. (3 seeds, 4 ep, CPU)	baseline (bare sum)	mean test-acc	0.272 (pstd 0.032)	4736
MNIST cls. (3 seeds, 4 ep, CPU)	upgraded	mean test-acc	0.303 (pstd 0.006)	5811
Clutter-MNIST det. toy (400 img, 8 ep)	baseline	final mAP50_proxy	0.022	—
Clutter-MNIST det. toy (400 img, 8 ep)	upgraded	final mAP50_proxy	0.085	—
Clutter-MNIST scale (2k img, 20 ep, GPU)	baseline	final mAP50_proxy	0.106	4821
Clutter-MNIST scale (2k img, 20 ep, GPU)	upgraded	final mAP50_proxy	0.135	5896

Table 1: Preliminary results. MNIST classification is over 3 paired seeds (mean, pstd); both detection rows are single-seed. The baseline detection “scale” value 0.106 is saturated by $\sim$ epoch 11; the upgraded “scale” value 0.135 is still climbing at epoch 20 (last three epoch values 0.117, 0.126, 0.135). Relative improvements: +11% mean on MNIST classification; $\sim 4\times$ on toy detection; +27% on scale detection.

Honest conclusions. We state the following plainly.

- The upgrade *consistently* beats the bare-sum aggregator across all three settings (MNIST classification, toy detection, scale detection).
- It does **not** overturn the vision falsification. Both detection numbers are below the prior config-F headline of 0.174 (measured at 5000 training images / 20 epochs); and on MNIST classification a trivial Linear(784, 10) reaches ~ 0.91 while these sit at ~ 0.30 .
- The per-image hypergraph construction (quadtree $\rightarrow$ stimulus graph $\rightarrow$ Forman–Ricci curvature $\rightarrow$ Hodge Laplacian) is CPU-bound, so the GPU did not materially speed training. A *topology cache* — reusing the per-image graph across epochs, since it is geometry-determined — is the planned enabler for the full 5000-image / 40-epoch run.
- The untested axis is *brain-predictivity* (Cichy-92 fMRI), not classification accuracy.

8 Related work

HyMeKo’s IR builds on hypergraph theory [1] and on the recent line of hypergraph neural networks, in particular HGNN [2]. The neural target draws on Kolmogorov–Arnold networks [3], and on discrete-geometric notions of curvature and flow: combinatorial / Forman Ricci curvature [4] and stochastic discrete Ricci flow rewiring (SDRF) [5]. The P-graph methodology follows the Friedler axioms [6]. The SysML lens targets the OMG SysML standard [7].

9 Conclusion and future work

The contribution of this work is infrastructure: a single canonical signed-typed hypergraph IR that compiles, under a validation gate and a structural-parity check, to many faithful targets, so

that a CPS model is one accountable object rather than N drifting artifacts. The empirical study is honest about its limits — the upgraded neural target consistently beats the falsified bare-sum aggregator, but does not overturn the standing vision falsification at the scales we can afford.

Future work follows three lines. First, the *topology cache* that turns the CPU-bound per-image hypergraph construction into a one-time cost, enabling the full 5000-image / 40-epoch tuned run. Second, using the P-graph SSG/ABB machinery (Section 6) as a principled, structure-aware search over architecture structure rather than ad-hoc tuning. Third, broadening the transform targets beyond those in Section 3 — candidate targets include BehaviorTree.CPP, PDDL, and ROS 2 actions — so that the same accountable IR can drive execution as well as analysis.

References

- [1] C. Berge. *Graphs and Hypergraphs*. North-Holland, 1973.
- [2] Y. Feng, H. You, Z. Zhang, R. Ji, Y. Gao. *Hypergraph Neural Networks*. AAAI, 2019.
- [3] Z. Liu et al. *KAN: Kolmogorov–Arnold Networks*. arXiv preprint, 2024.
- [4] R. Forman. *Bochner’s method for cell complexes and combinatorial Ricci curvature*. Discrete & Computational Geometry, 29(3):323–374, 2003.
- [5] J. Topping, F. Di Giovanni, B. P. Chamberlain, X. Dong, M. M. Bronstein. *Understanding over-squashing and bottlenecks on graphs via curvature*. ICLR, 2022.
- [6] F. Friedler, K. Tarján, Y. W. Huang, L. T. Fan. *Graph-theoretic approach to process synthesis: axioms and theorems*. Chemical Engineering Science, 1992.
- [7] Object Management Group. *OMG Systems Modeling Language (SysML)*. OMG specification.

Note on citations: bibliographic details follow the standard references for each method; exact pages, venues and identifiers should be verified against the project bib files before external use. No DOIs are asserted here.